

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Computer Vision and Image Processing (CSCS111/CSDS119)
Lab Assignment 4
Statistical and Order-Statistic Filtering

1 Aim and Objectives

1. To understand the mathematical formulation and operational principles of spatial domain statistical and order-statistic filters.
2. To implement non-linear order-statistic filters from scratch using vectorized NumPy operations:
 - Median Filter
 - Minimum (Erosion-like) Filter
 - Maximum (Dilation-like) Filter
 - Midpoint Filter
 - Alpha-Trimmed Mean Filter
 - Percentile / Ranked Order Filters (25th and 75th percentiles)
3. To inject synthetic Salt & Pepper (impulse) and Additive Gaussian noise into standard test imagery (*Cameraman*, 256×256).
4. To empirically analyze filter restoration capability across varying spatial neighborhood windows (3×3 , 5×5 , and 7×7) and investigate trade-offs between noise suppression and edge preservation.

2 Mathematical Background

Order-statistic filters are non-linear spatial filters whose response is based on ranking (ordering) the pixel intensities contained within a localized neighborhood window S_{xy} centered at (x, y) .

Let S_{xy} be a square spatial neighborhood of size $k \times k$, comprising $N = k^2$ pixels. Let the pixel values in S_{xy} be sorted in ascending order of intensity:

$$z_{(1)} \leq z_{(2)} \leq z_{(3)} \leq \dots \leq z_{(N)} \quad (1)$$

The filtered output $\hat{f}(x, y)$ is computed based on this ranked sequence:

1. **Median Filter:** Selects the median value of the ordered sequence:

$$\hat{f}_{\text{med}}(x, y) = \text{median}\{g(s, t) \mid (s, t) \in S_{xy}\} = z_{(\frac{N+1}{2})} \quad (2)$$

The median filter is effective against bipolar impulse (salt-and-pepper) noise because outlier impulse values reside in the extreme tails ($z_{(1)}$ or $z_{(N)}$) and are completely excluded from the output.

2. **Minimum Filter (Min Filter):** Selects the lowest intensity value:

$$\hat{f}_{\min}(x, y) = \min_{(s,t) \in S_{xy}} \{g(s, t)\} = z_{(1)} \quad (3)$$

This filter is effective for removing bright salt impulses (255), but broadens dark pepper pixels.

3. **Maximum Filter (Max Filter):** Selects the highest intensity value:

$$\hat{f}_{\max}(x, y) = \max_{(s,t) \in S_{xy}} \{g(s, t)\} = z_{(N)} \quad (4)$$

This filter eliminates dark pepper impulses (0), but broadens bright salt pixels.

4. **Midpoint Filter:** Computes the arithmetic average of the two extreme ranked intensities:

$$\hat{f}_{\text{mid}}(x, y) = \frac{1}{2} \left[\min_{(s,t) \in S_{xy}} \{g(s, t)\} + \max_{(s,t) \in S_{xy}} \{g(s, t)\} \right] = \frac{z_{(1)} + z_{(N)}}{2} \quad (5)$$

The midpoint filter combines order statistics with averaging, performing well for randomly distributed noise with short-tailed probability density functions (such as uniform noise).

5. **Alpha-Trimmed Mean Filter:** Trims $d = \lfloor \alpha N \rfloor$ extreme values from both ends of the sorted sequence and computes the arithmetic mean of the remaining $N - 2d$ values:

$$\hat{f}_{\alpha}(x, y) = \frac{1}{N - 2d} \sum_{i=d+1}^{N-d} z_{(i)}, \quad \text{where } 0 \leq \alpha < 0.5 \quad (6)$$

When $\alpha = 0$, the filter degenerates into an arithmetic mean filter. When $\alpha \rightarrow 0.5$ (specifically $d = \frac{N-1}{2}$), it becomes the median filter. It offers an adaptable balance for mixed noise (Gaussian plus impulse noise).

6. **Percentile Filter:** Replaces the center pixel with the intensity at a specified percentile $p \in [0, 100]$:

$$\hat{f}_p(x, y) = z_{(r)}, \quad \text{where } r = \left\lfloor \frac{p}{100}(N - 1) \right\rfloor + 1 \quad (7)$$

3 Implementation Details

The algorithm uses vectorized sliding-window extraction via `numpy.lib.stride_tricks.sliding_window` combined with reflection boundary padding (`mode='reflect'`). All neighborhood windows across the image grid are simultaneously extracted, reshaped into two-dimensional vectors per pixel, and sorted along the last axis (`axis=-1`).

3.1 Image Loading and Noise Generation

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Load Cameraman image
6 img = cv2.imread('Filtering-Cameraman.png', cv2.IMREAD_GRAYSCALE)
7 print(f"Loaded Cameraman image with dimensions: {img.shape}")
```

Loaded Cameraman image with dimensions: (256, 256)

```

1 # Noise injection utilities
2 def add_salt_and_pepper_noise(image, salt_prob=0.03, pepper_prob=0.03):
3     noisy = image.copy()
4     rnd = np.random.rand(*image.shape)
5     noisy[rnd < salt_prob] = 255
6     noisy[(rnd >= salt_prob) & (rnd < salt_prob + pepper_prob)] = 0
7     return noisy
8
9 def add_gaussian_noise(image, mean=0, sigma=20):
10     noisy = image.astype(np.float32) + np.random.normal(mean, sigma, image.shape)
11     return np.clip(noisy, 0, 255).astype(np.uint8)
12
13 np.random.seed(42)
14 noisy_sp = add_salt_and_pepper_noise(img)
15 noisy_gauss = add_gaussian_noise(img)
16
17 # Display original vs noisy inputs
18 fig, axes = plt.subplots(1, 3, figsize=(12, 4))
19 axes[0].imshow(img, cmap='gray')
20 axes[0].set_title("Original Cameraman")
21 axes[1].imshow(noisy_sp, cmap='gray')
22 axes[1].set_title("Salt & Pepper Noise (6%)")
23 axes[2].imshow(noisy_gauss, cmap='gray')
24 axes[2].set_title("Gaussian Noise (sigma=20)")
25 for ax in axes:
26     ax.axis('off')
27 plt.tight_layout()
28 plt.show()

```

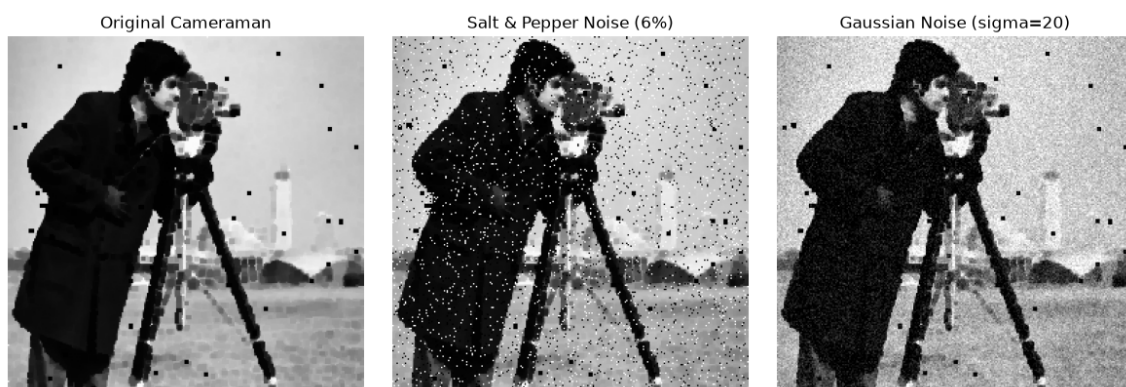


Figure 1: Original Cameraman test image and synthetically corrupted inputs: Salt & Pepper noise (6% total corruption) and Additive Gaussian noise ($\mu = 0, \sigma = 20$).

3.2 Statistical Order Filter Implementations

```

1 # Extract and sort sliding window neighborhoods (vectorized via NumPy)
2 def get_sorted_windows(image, k):
3     pad = k // 2
4     padded = np.pad(image, pad, mode='reflect')

```

```

5     windows = np.lib.stride_tricks.sliding_window_view(padded, (k, k))
6     return np.sort(windows.reshape(*image.shape, k * k), axis=-1)
7
8 # Statistical order filters
9 def median_filter(sw):
10     return sw[..., sw.shape[-1] // 2]
11
12 def min_filter(sw):
13     return sw[..., 0]
14
15 def max_filter(sw):
16     return sw[..., -1]
17
18 def midpoint_filter(sw):
19     return ((sw[..., 0].astype(float) + sw[..., -1].astype(float)) / 2).astype(np.
20         uint8)
21
22 def alpha_trimmed_mean_filter(sw, alpha=0.2):
23     d = max(1, int(alpha * sw.shape[-1])) # trim d elements from each tail
24     trimmed = sw[..., d : sw.shape[-1] - d]
25     return np.clip(trimmed.mean(axis=-1), 0, 255).astype(np.uint8)
26
27 def percentile_filter(sw, p):
28     idx = int(np.round((p / 100.0) * (sw.shape[-1] - 1)))
29     return sw[..., idx]

```

4 Experimental Results

4.1 Comparison of Filters on Salt & Pepper Noise (3×3 Kernel)

The initial evaluation applies all order-statistic filters to the 6% Salt & Pepper degraded image using a standard 3×3 window ($k = 3, N = 9$).

```

1 # Apply each filter on Salt & Pepper noisy image (3x3 neighborhood)
2 k = 3
3 sw_sp3 = get_sorted_windows(noisy_sp, k)
4
5 filters_3x3 = {
6     "Median": median_filter(sw_sp3),
7     "Min": min_filter(sw_sp3),
8     "Max": max_filter(sw_sp3),
9     "Midpoint": midpoint_filter(sw_sp3),
10    "Alpha-Trimmed (alpha=0.2)": alpha_trimmed_mean_filter(sw_sp3, alpha=0.2),
11    "25th Percentile": percentile_filter(sw_sp3, 25),
12    "75th Percentile": percentile_filter(sw_sp3, 75)
13 }
14
15 fig, axes = plt.subplots(2, 4, figsize=(16, 8))
16 axes = axes.ravel()
17
18 axes[0].imshow(noisy_sp, cmap='gray')
19 axes[0].set_title("Noisy Input (Salt & Pepper)")
20 axes[0].axis('off')
21
22 for i, (name, filtered_img) in enumerate(filters_3x3.items(), start=1):

```

```

23 axes[i].imshow(filtered_img, cmap='gray')
24 axes[i].set_title(f"{name} (3x3)")
25 axes[i].axis('off')
26
27 plt.tight_layout()
28 plt.show()

```



Figure 2: Comparative performance of statistical order filters on 6% Salt & Pepper noise with a 3×3 neighborhood window.

4.2 Impact of Spatial Neighborhood Sizes (3×3 , 5×5 , and 7×7)

```

1 # Evaluate filters across neighborhood sizes 3x3, 5x5, and 7x7
2 sizes = [3, 5, 7]
3 filter_funcs = {
4     "Median": median_filter,
5     "Midpoint": midpoint_filter,
6     "Alpha-Trimmed": lambda sw: alpha_trimmed_mean_filter(sw, alpha=0.2),
7     "25th Percentile": lambda sw: percentile_filter(sw, 25),
8     "75th Percentile": lambda sw: percentile_filter(sw, 75)
9 }
10
11 fig, axes = plt.subplots(len(filter_funcs), len(sizes), figsize=(12, 3.2 * len(
12     filter_funcs)))
13
14 for r, (fname, ffunc) in enumerate(filter_funcs.items()):
15     for c, sz in enumerate(sizes):
16         sw = get_sorted_windows(noisy_sp, sz)
17         res = ffunc(sw)
18         axes[r, c].imshow(res, cmap='gray')
19         axes[r, c].set_title(f"{fname} ({sz}x{sz})")
20         axes[r, c].axis('off')
21
22 plt.tight_layout()
23 plt.show()

```

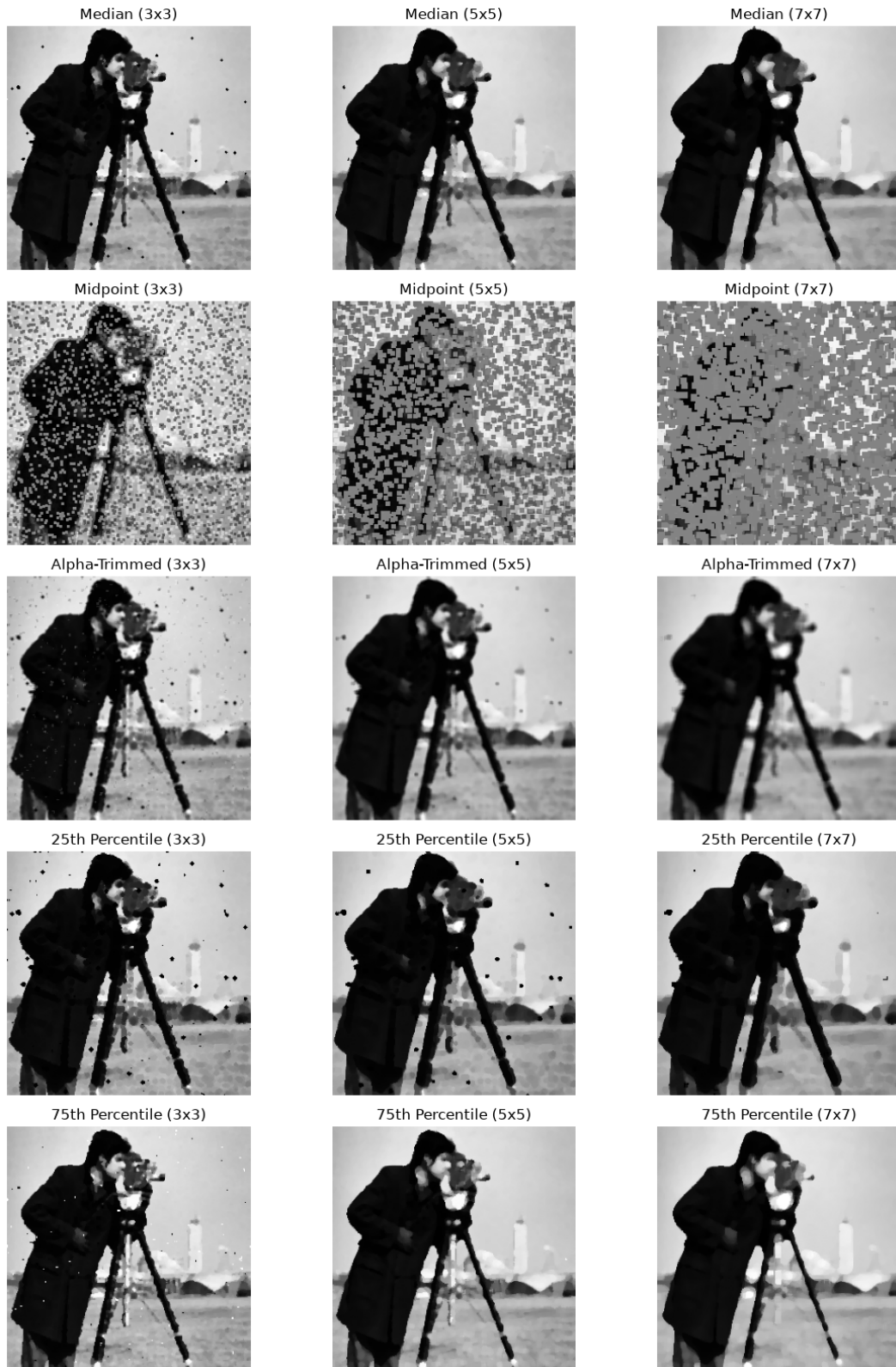


Figure 3: Effect of neighborhood window sizes (3×3 , 5×5 , 7×7) on noise removal efficiency versus edge detail preservation.

4.3 Evaluation on Additive Gaussian Noise

To contrast the filtering characteristics between impulse and continuous Gaussian degradation, we evaluate the Median Filter, Midpoint Filter, and Alpha-Trimmed Mean Filter ($\alpha = 0.2$) on the Gaussian corrupted image with $k = 5$.

```

1 # Comparing filter performance on Gaussian noise (5x5 neighborhood)
2 k = 5
3 sw_g5 = get_sorted_windows(noisy_gauss, k)
4
5 gauss_results = {
6     "Noisy Gaussian Input": noisy_gauss,
7     "Median Filter (5x5)": median_filter(sw_g5),
8     "Midpoint Filter (5x5)": midpoint_filter(sw_g5),
9     "Alpha-Trimmed (5x5, a=0.2)": alpha_trimmed_mean_filter(sw_g5, alpha=0.2)
10 }
11
12 fig, axes = plt.subplots(1, 4, figsize=(16, 4))
13 for ax, (title, out_img) in zip(axes, gauss_results.items()):
14     ax.imshow(out_img, cmap='gray')
15     ax.set_title(title)
16     ax.axis('off')
17
18 plt.tight_layout()
19 plt.show()

```



Figure 4: Performance comparison on Additive Gaussian noise ($\sigma = 20$) using a 5×5 neighborhood window.

5 Comparative Summary Table

Table 1 provides a structured synthesis of the theoretical definitions, computational characteristics, optimal applications, and visual behaviors of the investigated statistical order filters.

Filter	Mathematical Formula / Rank Index	Complexity	Optimal Noise Environment	Observed Output Characteristics
Median	$z_{(\frac{N+1}{2})}$	$O(N \log N)$	Salt & Pepper (Impulse) Noise	Completely eliminates isolated impulses; preserves sharp boundaries and fine contours.
Min (Erosion)	$z_{(1)}$	$O(N)$	Salt Noise (pure positive spikes)	Eliminates bright pixels (255); enlarges dark regions and accentuates pepper noise.
Max (Dilation)	$z_{(N)}$	$O(N)$	Pepper Noise (pure negative spikes)	Eliminates dark pixels (0); expands highlights and bright noise structures.
Midpoint	$\frac{z_{(1)} + z_{(N)}}{2}$	$O(N)$	Short-tailed / Uniform Noise	Blurs heavily under impulse noise because outliers dominate extreme ranks.
Alpha-Trimmed Mean	$\frac{1}{N-2d} \sum_{i=d+1}^{N-d} z_{(i)}$	$O(N \log N)$	Mixed Noise (Gaussian + Impulse)	Provides tunable trade-off between outlier rejection and Gaussian noise smoothing.
25th Percentile	$z_{(\lfloor 0.25(N-1) \rfloor + 1)}$	$O(N \log N)$	Salt-dominant Noise	Biases intensities downward; suppresses bright impulses while darkening the background.
75th Percentile	$z_{(\lfloor 0.75(N-1) \rfloor + 1)}$	$O(N \log N)$	Pepper-dominant Noise	Biases intensities upward; suppresses dark impulses while brightening the background.

Table 1: Comparative analysis of spatial statistical and order-statistic filters ($N = k^2$).

6 Observations and In-Depth Analysis

6.1 Comparison of Statistical Order Filtering Methods

1. **Median Filter:** Demonstrates superior noise suppression for ****salt-and-pepper noise****. Because impulse noise creates extreme outliers at the boundaries of the sorted distribution, the median order index ($z_{((N+1)/2)}$) ignores these extreme values entirely. Fine structural details (such as the tripod legs, coat silhouette, and horizon line in the Cameraman image) remain sharp without the edge blurring seen in linear averaging filters.
2. **Min and Max Filters:**
 - *Min Filter:* Replaces each pixel with the minimum neighborhood value $z_{(1)}$. This eliminates bright salt noise (255), but causes dark pepper impulses to dilate significantly, creating dark blotches and a general darkening of the image.

- *Max Filter*: Replaces each pixel with the maximum neighborhood value $z_{(N)}$. This eliminates dark pepper noise (0), but causes bright salt noise to dilate, creating bright spots and an overall brightening of the image.
3. **Midpoint Filter**: Evaluates $\frac{z_{(1)} + z_{(N)}}{2}$. The filter relies on extreme values. While suitable for short-tailed symmetric distributions (such as uniform noise), it degrades severely in the presence of salt-and-pepper noise because outliers dominate either $z_{(1)}$, $z_{(N)}$, or both, leading to visible halo artifacts around impulse locations.
 4. **Alpha-Trimmed Mean Filter**: Provides an adjustable parameter $\alpha \in [0, 0.5)$. Trimming the extreme $d = \lfloor \alpha N \rfloor$ outliers from both tails removes impulse spikes, while averaging the remaining central values smooths Gaussian background fluctuations. On additive Gaussian noise, it yields a smoother visual output than the standard median filter while avoiding the boundary smearing of arithmetic mean filtering.
 5. **Ranked / Percentile Filters (25th vs 75th Percentile)**:
 - The 25th percentile filter selects a rank below the median, biasing pixel values downward. This suppresses bright salt impulses while slightly darkening intermediate background tones.
 - The 75th percentile filter selects a rank above the median, biasing pixel values upward. This suppresses dark pepper impulses while slightly brightening midtones.

6.2 Impact of Spatial Neighborhood Size (3×3 , 5×5 , 7×7)

1. **3×3 Window ($N = 9$)**: Provides the best trade-off between impulse noise suppression and high-frequency edge preservation for moderate noise densities ($\leq 10\%$). High-contrast edges and thin lines remain intact.
2. **5×5 Window ($N = 25$)**: Increases noise suppression capacity, removing small impulse clusters that survive a 3×3 window. However, subtle textural nuances and sharp corner profiles begin to soften.
3. **7×7 Window ($N = 49$)**: Removes severe noise densities, but introduces significant geometric distortions. Thin line features (such as the legs of the tripod) are eroded or blurred, corners are rounded, and the overall image resolution degrades.